Union Types

If a function accepts a range of types for a particular parameter or returns a range of types, it is possible to declare the set of valid types using each type keyword separated by the pipe symbol |.

Imagine we have a function that can work with either `int` or `string` data. To declare the union type, we'd use `function someFunction(int|string $input)`. The order of types does not matter; `string|int` would also work, but repeating a type is fatal. `int|string|int` in our code would result in the following message in the error log: `Duplicate type int is redundant`. The following code demonstrates a union type in action.

```
function itsComplicated(int|string $in): string
{
    switch (gettype($in)) {
        case "string":
            return "You gave me a string";
            break;
        case "integer":
            return "You gave me an integer";
            break;
    }
}

print itsComplicated("string") . PHP_EOL;
print itsComplicated(1) . PHP_EOL;
```

```
Output:
You gave me a string
You gave me an integer
```

The function `itsComplicated()` has a single argument – `$in`. Its type is either `string` or `int`, so we declare a union type of `string|int`. The rest of the code is straightforward: we use the built-in `gettype()` to check the data type of the incoming data and go from there. Using a union in a return type declaration is no different: